INDIAN INSTITUTE OF EMBEDDED SYSTEMS

Koramangala, Bengaluru.

PROJECT REPORT ON

“Library Management System Using C Programming”

Submitted by:

B R MAHIDAR

S NAVEEN KUMAR

DATE: 08-04-2026

TABLE OF CONTENTS

Introduction

A Library Management System is designed to manage and organize the daily operations of a library efficiently. In traditional libraries, maintaining records of books, students, and transactions manually can be time-consuming and prone to errors. To overcome these issues, this project implements a simple and effective Library Management System using the C programming language.

The system uses file handling techniques to store and retrieve data from CSV files, eliminating the need for complex databases. It provides functionalities such as adding and managing books, maintaining student records, issuing and returning books, and tracking transactions. The project focuses on simplicity, ease of use, and practical implementation of core programming concepts, making it suitable for academic learning and real-world applications.

Abstract

The application provides secure access through a login mechanism and offers multiple functionalities including adding, displaying, searching, and deleting books. It also enables the management of student records and supports issuing and returning of books with manual date entry. All transactions are recorded systematically, allowing easy tracking of issued and returned books.

The system ensures data persistence using external files such as books.csv, students.csv, and transactions.csv, making it lightweight and easy to maintain without the need for a database. Additionally, it demonstrates the use of fundamental programming concepts such as file handling, structures, conditional statements, and modular programming.

Objectives

The main objectives of the Library Management System are:

- To develop a simple and efficient system to manage library operations using the C programming language.
- To store and manage book, students, and transaction data using file handling (CSV files).
- To provide functionalities such as adding, displaying, searching, and deleting books.
- To maintain student records in an organized manner.
- To implement book issuing and returning operations with proper tracking.
- To reduce manual work and minimize errors in maintaining records.
- To demonstrate the use of core programming concepts like file handling, functions, and modular programming.

Project Details

The Library Management System is a console-based application developed using the C programming language to automate and simplify the management of library operations. The system is designed to handle key functionalities such as book management, student management, and transaction tracking in an efficient and structured manner.

The application uses file handling to store data in CSV (Comma-Separated Values) files, namely books.csv, students.csv, and transactions.csv. This ensures that all records are permanently stored and can be retrieved whenever required, eliminating the need for a database.

The system begins with a login module that provides basic security by allowing only authorized users to access the application. After successful authentication, a menu-driven interface is displayed, enabling users to choose different operations easily.

The Book Management module includes functionalities such as adding new books, displaying available books, searching for a specific book using its ID, and deleting a book record. These operations are performed using file read and write operations, ensuring that data is updated dynamically.

The Transaction Management module is a key component of the system. It handles issuing and returning books. When a book is issued, the system records the student ID, book ID, issue date, and marks the status as "NOT_RETURNED". During return, the system updates the record with the return date and changes the status accordingly. It also demonstrates a simple fine calculation mechanism for late returns.

The system uses modular programming, where each functionality is implemented as a separate function, improving readability, maintainability, and reusability of code. It also demonstrates important concepts such as file handling, conditional statements, loops, and string manipulation.

Overall, this project provides a practical implementation of a real-world library system in a simplified manner. It serves as a strong foundation for future enhancements such as graphical user interface (GUI), database integration (like MySQL), user roles, advanced search features, and automated fine calculation.

Module Implementation

- **login()**

The `login()` function is used to authenticate the user before accessing the system. It takes the username and password as input and compares them with predefined credentials using `strcmp()`. If both the username and password match, it returns 1, allowing access to the system. Otherwise, it returns 0 and denies access.

- **add Book()**

The `add Book()` function is used to add new book records into the system. It takes book ID, name, author, and quantity as input from the user. The data is stored in the `books.csv` file using append mode, ensuring that existing data is not overwritten. The information is saved in CSV format using `fprintf()`.

- **display Books()**

The `display Books()` function displays all the books available in the library. It reads data from the `books.csv` file using `fscanf()` in a loop until the end of the file is reached. The details of each book are printed in a formatted table, making it easy for the user to view all records.

- **search Book()**

The `search Book()` function is used to find a specific book using its ID. It takes the book ID as input and searches through the file line by line. If a matching record is found, the book details are displayed. If no match is found, it shows a "Book not found" message.

- **delete Book()**

The `delete Book()` function removes a book record from the system. It uses a temporary file to store all records except the one to be deleted. After copying, the original file is deleted and the temporary file is renamed. This method ensures safe and efficient deletion of records.

- **Add Student()**

The `add Student ()` function is used to add student details into the system. It takes student ID and name as input and stores the data in the `students.csv` file. The file is opened in append mode to preserve existing records, and data is stored in CSV format.

- **Issue Book()**

The `issue Book () ` function records the issuing of a book to a student. It takes student ID, book ID, and issue date as input. The details are stored in the `transactions.csv` file along with the status "NOT_RETURNED". This helps in tracking issued books.

- **Return Book()**

The `return Book () ` function updates the status of a returned book. It searches for the matching transaction using student ID and book ID. If the book is not already returned, it updates the return date and writes the updated record to a temporary file. The original file is then replaced.

- **Display Transactions()**

The `display Transactions () ` function displays all the transaction records. It reads data from the `transactions.csv` file and shows details such as student ID, book ID, issue date, and return date. This helps in tracking all library activities efficiently.

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Function declarations
int login();
void addBook();
void displayBooks();
void searchBook();
void deleteBook();
void addStudent();
void issueBook();
void returnBook();
void displayTransactions();

// ================= MAIN =================
int main() {
   int choice;
```

```
if (!login()) {
    printf("Access Denied!\n");
    return 0;
}
while (1) {
    printf("\n================================\n");
    printf("   LIBRARY MANAGEMENT SYSTEM\n");
    printf("================================\n");
    printf("1. Add Book\n");
    printf("2. Display Books\n");
    printf("3. Search Book\n");
    printf("4. Delete Book\n");
    printf("5. Add Student\n");
    printf("6. Issue Book\n");
    printf("7. Return Book\n");
    printf("8. Display Transactions\n");
    printf("9. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: addBook(); break;
        case 2: displayBooks(); break;
        case 3: searchBook(); break;
        case 4: deleteBook(); break;
        case 5: addStudent(); break;
        case 6: issueBook(); break;
        case 7: returnBook(); break;
        case 8: displayTransactions(); break;
```

```
            case 9: exit(0);
            default: printf("Invalid choice!\n");
        }
    }
}

// ================== LOGIN ==================
int login() {
    char user[20], pass[20];
    printf("Username: ");
    scanf("%s", user);
    printf("Password: ");
    scanf("%s", pass);
    if (strcmp(user, "admin") == 0 && strcmp(pass, "1234") == 0)
        return 1;
    return 0;
}

// ================== ADD BOOK ==================
void addBook() {
    FILE *fp = fopen("books.csv", "a");
    int id, qty;
    char name[50], author[50];
    printf("Enter Book ID: ");
    scanf("%d", &id);
    printf("Enter Book Name: ");
    scanf(" %[^\n]", name);
    printf("Enter Author: ");
    scanf(" %[^\n]", author);
```

```
    printf("Enter Quantity: ");
    scanf("%d", &qty);
    fprintf(fp, "%d,%s,%s,%d\n", id, name, author, qty);
    fclose(fp);
    printf("Book Added Successfully!\n");
}

// ================== DISPLAY BOOKS ==================
void displayBooks() {
    FILE *fp = fopen("books.csv", "r");
    int id, qty;
    char name[50], author[50];
    printf("\nID\tName\t\tAuthor\t\tQty\n");
    printf("-----------------------------------------\n");
    while (fscanf(fp, "%d,%[^,],%[^,],%d\n",&id, name, author, &qty) != EOF) {
        printf("%d\t%s\t\t%s\t\t%d\n", id, name, author, qty);
    }
    fclose(fp);
}

// ================== SEARCH BOOK ==================
void searchBook() {
    FILE *fp = fopen("books.csv", "r");
    int id, qty, searchId, found = 0;
    char name[50], author[50];
    printf("Enter Book ID to search: ");
    scanf("%d", &searchId);
    while (fscanf(fp, "%d,%[^,],%[^,],%d\n",&id, name, author, &qty) != EOF) {
        if (id == searchId) {
```

```
            printf("\nBook Found!\n");
            printf("ID: %d\nName: %s\nAuthor: %s\nQty: %d\n",
                id, name, author, qty);
            found = 1;
            break;
        }
    }

    if (!found)
        printf("Book not found!\n");
    fclose(fp);
}

// ================== DELETE BOOK ==================
void deleteBook() {
    FILE *fp = fopen("books.csv", "r");
    FILE *temp = fopen("temp.csv", "w");
    int id, qty, deleteId, found = 0;
    char name[50], author[50];
    printf("Enter Book ID to delete: ");
    scanf("%d", &deleteId);
    while (fscanf(fp, "%d,%[^,],%[^,],%d\n",&id, name, author, &qty) != EOF) {
        if (id == deleteId) {
            found = 1;
            printf("Book Deleted Successfully!\n");
            continue;
        }
        fprintf(temp, "%d,%s,%s,%d\n", id, name, author, qty);
    }
```

```
    fclose(fp);
    fclose(temp);
    remove("books.csv");
    rename("temp.csv", "books.csv");
    if (!found)
        printf("Book not found!\n");
}

// ================== ADD STUDENT ==================
void addStudent() {
    FILE *fp = fopen("students.csv", "a");
    int id;
    char name[50];
    printf("Enter Student ID: ");
    scanf("%d", &id);
    printf("Enter Student Name: ");
    scanf(" %[^\n]", name);
    fprintf(fp, "%d,%s\n", id, name);
    fclose(fp);
    printf("Student Added Successfully!\n");
}

// ================== ISSUE BOOK ==================
void issueBook() {
    FILE *fp = fopen("transactions.csv", "a");
    int sid, bid;
    char date[20];
    printf("Enter Student ID: ");
    scanf("%d", &sid);
```

```
    printf("Enter Book ID: ");
    scanf("%d", &bid);
    printf("Enter Issue Date (DD-MM-YY): ");
    scanf("%s", date);
    fprintf(fp, "%d,%d,%s,NOT_RETURNED\n", sid, bid, date);
    fclose(fp);

    printf("Book Issued Successfully!\n");
}

// ================= RETURN BOOK =================
void returnBook() {
    FILE *fp = fopen("transactions.csv", "r");
    FILE *temp = fopen("temp.csv", "w");
    int sid, bid;
    char rdate[20];
    printf("Enter Student ID: ");
    scanf("%d", &sid);
    printf("Enter Book ID: ");
    scanf("%d", &bid);
    printf("Enter Return Date (DD-MM-YY): ");
    scanf("%s", rdate);
    int s, b;
    char idate[20], ret[20];
    while (fscanf(fp, "%d,%d,%[^,],%s\n",&s, &b, idate, ret) != EOF) {
        if (s == sid && b == bid && strcmp(ret, "NOT_RETURNED") == 0) {
            fprintf(temp, "%d,%d,%s,%s\n", s, b, idate, rdate);
            printf("\nBook Returned Successfully!\n");
            printf("Fine: Rs. 20 (Demo)\n");
```

```
        } else {
            fprintf(temp, "%d,%d,%s,%s\n", s, b, idate, ret);
        }
    }
    fclose(fp);
    fclose(temp);

    remove("transactions.csv");
    rename("temp.csv", "transactions.csv");
}

// ================== DISPLAY TRANSACTIONS ==================
void displayTransactions() {
    FILE *fp = fopen("transactions.csv", "r");
    int sid, bid;
    char issue[20], ret[20];
    printf("\nStudentID\tBookID\tIssueDate\tReturnDate\n");
    printf("-----------------------------------------------------------\n");
    while (fscanf(fp, "%d,%d,%[^,],%s\n",&sid, &bid, issue, ret) != EOF)
printf("%d\t\t%d\t%s\t\t%s\n", sid, bid, issue, ret);
    }
    fclose(fp);
}
```

Output

- **Output Menu**

```
mahi@ubuntu:~/LibraryProject$ ./a.out
Username: admin
Password: 1234

==============================
   LIBRARY MANAGEMENT SYSTEM
==============================
1. Add Book
2. Display Books
3. Search Book
4. Delete Book
5. Add Student
6. Issue Book
7. Return Book
8. Display Transactions
9. Exit
Enter choice:
```

- **Add Book**

```
Enter choice: 1
Enter Book ID: 3
Enter Book Name: Learn C the Hard Way
Enter Author: Zed A. Shaw
Enter Quantity: 6
Book Added Successfully!
```

- **Issue Book**

```
Enter choice: 6
Enter Student ID: 11
Enter Book ID: 3
Enter Issue Date (DD-MM-YY): 08-04-26
Book Issued Successfully!
```

- **Add Student**

```
Enter choice: 5
Enter Student ID: 82
Enter Student Name: Naveen
Student Added Successfully!
```

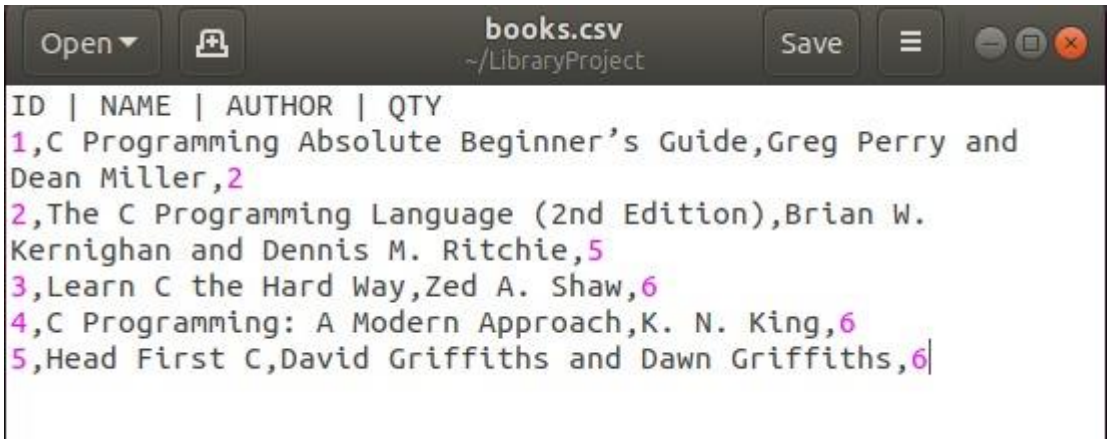
- **Return Book**

```
Enter choice: 7
Enter Student ID: 11
Enter Book ID: 3
Enter Return Date (DD-MM-YY): 14-04-26

Book Returned Successfully!
Fine: Rs. 20
 Late Submisission
```

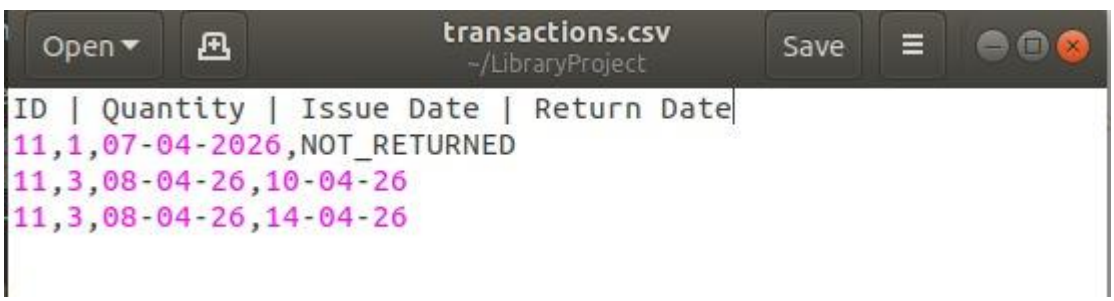
- **All transactions**

```
Enter choice: 8

StudentID       BookID  IssueDate       ReturnDate
-------------------------------------------------------
11              1       07-04-2026              NOT_RETURNED
11              3       08-04-26                10-04-26
11              3       08-04-26                14-04-26
```

- **books.csv**



- **transactions.csv**



Conclusion

The Library Management System developed using C successfully demonstrates how programming concepts can be applied to manage real-world problems efficiently. The system provides functionalities such as adding, searching, deleting books, managing student records, and tracking transactions using file handling. It reduces manual effort, improves accuracy, and ensures organized data storage using CSV files. Overall, the project is simple, effective, and serves as a strong foundation for understanding file-based systems and modular programming.

Future Scope

The system can be further improved by adding advanced features and modern technologies.

- Development of a Graphical User Interface (GUI) for better user experience.
- Feature to check real-time book availability status.
- Conversion into a web-based or mobile application for remote access.
- Implementation of advanced search filters (by author, name, category).
- Adding report generation for transactions and library usage.